Configuración de Parámetros UserParameter — Zabbix Agent 2

Empresa:	EIT Logística S.A.
Fecha:	11 de noviembre de 2026
Implementado por:	Felipe Ramos — Supervisor de TI
Plataforma:	Zabbix 8.0.0 (Agent 2) sobre Windows Server

---

1. Objetivo

Implementar monitoreo personalizado mediante **UserParameter** en Zabbix Agent 2 para detectar qué **usuarios** y **procesos** consumen más recursos (CPU y RAM) en los servidores de producción de EIT Logística.

El template estándar "Windows by Zabbix agent" no incluye monitoreo por proceso ni por usuario. Esta solución agrega esa capacidad mediante scripts PowerShell ejecutados por el agente.

2. Servidores configurados

Servidor	Rol	Agente	Estado
EITAPP	Servidor de aplicaciones	Zabbix Agent 2 (8.0.0)	Configurado
EITWEB	Servidor web	Zabbix Agent 2 (8.0.0)	Configurado
EITAPPPUIG	Servidor de aplicaciones (Puig)	Zabbix Agent 2	Pendiente

3. Creación de carpetas y estructura de archivos

3.1. Crear la carpeta de scripts

En cada servidor, abrir PowerShell como Administrador y ejecutar:

```
New-Item -ItemType Directory -Path "C:\Zabbix\Scripts" -Force
```

3.2. Estructura resultante

```
C:\Zabbix\Scripts\
  top_cpu_processes.ps1      # Top 10 procesos por consumo de CPU
  top_mem_processes.ps1      # Top 10 procesos por consumo de RAM
  user_cpu_usage.ps1         # Consumo de CPU agrupado por usuario
  user_mem_usage.ps1         # Consumo de RAM agrupado por usuario
```

3.3. Crear cada script

Ejecutar los siguientes comandos en PowerShell (como Administrador) para crear cada archivo:

top_cpu_processes.ps1:

```
@'
Get-Process | Sort-Object CPU -Descending |
  Select-Object -First 10 | ForEach-Object {
    '{0}|CPU:{1:N1}|RAM:{2:N0}MB' -f
      $_.ProcessName, $_.CPU, ($_.WorkingSet64/1MB)
  } | Out-String
'@ | Set-Content "C:\Zabbix\Scripts\top_cpu_processes.ps1" -Encoding ASCII
```

top_mem_processes.ps1:

```
@'
Get-Process | Sort-Object WorkingSet64 -Descending |
  Select-Object -First 10 | ForEach-Object {
    '{0}|RAM:{1:N0}MB|PID:{2}' -f
      $_.ProcessName, ($_.WorkingSet64/1MB), $_.Id
  } | Out-String
'@ | Set-Content "C:\Zabbix\Scripts\top_mem_processes.ps1" -Encoding ASCII
```

user_cpu_usage.ps1:

```
@'
$data = Get-Process -IncludeUserName -ErrorAction SilentlyContinue |
  Where-Object { $_.UserName } |
  Group-Object UserName | ForEach-Object {
    [PSCustomObject]@{
      User = $_.Name
      CPU  = ($_.Group | Measure-Object CPU -Sum).Sum
      RAM  = ($_.Group | Measure-Object WorkingSet64 -Sum).Sum/1MB
      Procs = $_.Count
    }
  } | Sort-Object CPU -Descending
$data | ForEach-Object {
    '{0}|CPU:{1:N1}|RAM:{2:N0}MB|Procs:{3}' -f
      $_.User, $_.CPU, $_.RAM, $_.Procs
  } | Out-String
'@ | Set-Content "C:\Zabbix\Scripts\user_cpu_usage.ps1" -Encoding ASCII
```

user_mem_usage.ps1:

```
@'
$data = Get-Process -IncludeUserName -ErrorAction SilentlyContinue |
  Where-Object { $_.UserName } |
  Group-Object UserName | ForEach-Object {
    [PSCustomObject]@{
      User = $_.Name
      RAM  = ($_.Group | Measure-Object WorkingSet64 -Sum).Sum/1MB
      CPU  = ($_.Group | Measure-Object CPU -Sum).Sum
      Procs = $_.Count
    }
  } | Sort-Object RAM -Descending
$data | ForEach-Object {
    '{0}|RAM:{1:N0}MB|CPU:{2:N1}|Procs:{3}' -f
      $_.User, $_.RAM, $_.CPU, $_.Procs
  } | Out-String
'@ | Set-Content "C:\Zabbix\Scripts\user_mem_usage.ps1" -Encoding ASCII
```

3.4. Verificar que los scripts se crearon correctamente

```
Get-ChildItem "C:\Zabbix\Scripts\*.ps1" | Select-Object Name, Length, LastWriteTime
```

3.5. Probar cada script manualmente

```
powershell -NoProfile -ExecutionPolicy Bypass -File "C:\Zabbix\Scripts\top_cpu_processes.ps1"
powershell -NoProfile -ExecutionPolicy Bypass -File "C:\Zabbix\Scripts\user_cpu_usage.ps1"
```

4. Scripts PowerShell

4.1. top_cpu_processes.ps1

Devuelve los 10 procesos con mayor consumo de CPU acumulado.

```
Get-Process | Sort-Object CPU -Descending |
  Select-Object -First 10 | ForEach-Object {
    '{0}|CPU:{1:N1}|RAM:{2:N0}MB' -f
      $_.ProcessName, $_.CPU, ($_.WorkingSet64/1MB)
  } | Out-String
```

Salida de ejemplo:

```
perfmon|CPU:325.595,3|RAM:74MB
ServerManager|CPU:126.620,1|RAM:195MB
WMSTekCore|CPU:65.326,3|RAM:191MB
svchost|CPU:52.520,6|RAM:42MB
```

4.2. top_mem_processes.ps1

Devuelve los 10 procesos con mayor consumo de memoria RAM.

```
Get-Process | Sort-Object WorkingSet64 -Descending |
  Select-Object -First 10 | ForEach-Object {
    '{0}|RAM:{1:N0}MB|PID:{2}' -f
      $_.ProcessName, ($_.WorkingSet64/1MB), $_.Id
  } | Out-String
```

4.3. user_cpu_usage.ps1

Agrupa todos los procesos por usuario del sistema y muestra el consumo acumulado de CPU, ordenado de mayor a menor.

> **Nota:** Se usa Sort-Object sobre el objeto antes de formatear, para evitar problemas con el formato de números en locale español (coma como separador decimal).

```
$data = Get-Process -IncludeUserName -ErrorAction SilentlyContinue |
  Where-Object { $_.UserName } |
  Group-Object UserName | ForEach-Object {
    [PSCustomObject]@{
      User = $_.Name
      CPU  = ($_.Group | Measure-Object CPU -Sum).Sum
      RAM  = ($_.Group | Measure-Object WorkingSet64 -Sum).Sum/1MB
      Procs = $_.Count
    }
  } | Sort-Object CPU -Descending
$data | ForEach-Object {
    '{0}|CPU:{1:N1}|RAM:{2:N0}MB|Procs:{3}' -f
      $_.User, $_.CPU, $_.RAM, $_.Procs
  } | Out-String
```

Salida de ejemplo (EITAPP):

```
EIT\tekadmin|CPU:454.905,6|RAM:1.119MB|Procs:18
NT AUTHORITY\SYSTEM|CPU:151.018,7|RAM:2.527MB|Procs:33
EIT\gsepulveda|CPU:4.660,5|RAM:426MB|Procs:11
EIT\stgadmin|CPU:2.965,9|RAM:323MB|Procs:10
EIT\framos|CPU:106,8|RAM:654MB|Procs:13
```

4.4. user_mem_usage.ps1

Agrupa todos los procesos por usuario y muestra el consumo acumulado de RAM, ordenado de mayor a menor.

```
$data = Get-Process -IncludeUserName -ErrorAction SilentlyContinue |
  Where-Object { $_.UserName } |
  Group-Object UserName | ForEach-Object {
    [PSCustomObject]@{
      User = $_.Name
      RAM  = ($_.Group | Measure-Object WorkingSet64 -Sum).Sum/1MB
      CPU  = ($_.Group | Measure-Object CPU -Sum).Sum
      Procs = $_.Count
    }
  } | Sort-Object RAM -Descending
$data | ForEach-Object {
    '{0}|RAM:{1:N0}MB|CPU:{2:N1}|Procs:{3}' -f
      $_.User, $_.RAM, $_.CPU, $_.Procs
  } | Out-String
```

5. Configuración del agente — UserParameter

Se agregaron las siguientes líneas al final del archivo de configuración de cada servidor:

Archivo: `C:\Program Files\Zabbix Agent 2\zabbix_agent2.conf`

```
# === Monitoreo de procesos y usuarios - EIT Logistica ===
UserParameter=custom.top.cpu.processes,powershell -NoProfile
    -ExecutionPolicy Bypass -File "C:\Zabbix\Scripts\top_cpu_processes.ps1"
UserParameter=custom.top.mem.processes,powershell -NoProfile
    -ExecutionPolicy Bypass -File "C:\Zabbix\Scripts\top_mem_processes.ps1"
UserParameter=custom.user.cpu.usage,powershell -NoProfile
    -ExecutionPolicy Bypass -File "C:\Zabbix\Scripts\user_cpu_usage.ps1"
UserParameter=custom.user.mem.usage,powershell -NoProfile
    -ExecutionPolicy Bypass -File "C:\Zabbix\Scripts\user_mem_usage.ps1"
```

Después de agregar las líneas, se reinició el servicio:

```
Restart-Service "Zabbix Agent 2"
```

6. Items a crear en Zabbix web UI

Estos items deben crearse en el template **"Windows by Zabbix agent"** para que se propaguen a todos los servidores automáticamente.

Ruta: Configuration → Templates → Windows by Zabbix agent → Items → Create item

#	Name	Key	Info	Interval
1	Top CPU consuming processes on {HOST.NAME}	`custom.top.cpu.processes`	Text	5m
2	Top memory consuming processes on {HOST.NAME}	`custom.top.mem.processes`	Text	5m
3	CPU usage by user on {HOST.NAME}	`custom.user.cpu.usage`	Text	5m
4	Memory usage by user on {HOST.NAME}	`custom.user.mem.usage`	Text	5m

Importante: El campo "Type" debe ser **"Zabbix agent"** (no "Zabbix agent (active)"). El "Type of information" es **"Text"** porque devuelven cadenas de texto.

7. Verificación

Desde el servidor Zabbix (Linux):

```
zabbix_get -s <IP_DEL_SERVIDOR> -k custom.top.cpu.processes
zabbix_get -s <IP_DEL_SERVIDOR> -k custom.user.cpu.usage
```

Desde Zabbix web UI:

- Ir a **Monitoring** → **Latest data**
- Filtrar por host (EITAPP, EITWEB o EITAPPPUIG)

- Buscar "custom." o "Top CPU"

8. Problemas encontrados y soluciones

8.1. Error de Sort-Object con locale español

Problema: El script original intentaba parsear el string formateado para ordenar, pero el formato de números en locale español usa coma como separador decimal (ej: 1.237,4), causando error al convertir a Double.

Solución: Se reestructuró el script para ordenar sobre el objeto PSCustomObject antes de aplicar el formato de texto.

8.2. UserParameters duplicados en EITWEB

Problema: Al ejecutar Add-Content dos veces, se duplicaron las 4 líneas de UserParameter. Zabbix Agent 2 no permite keys duplicadas y falla al iniciar.

Solución: Se eliminaron las líneas duplicadas y se reescribió el archivo sin BOM usando UTF8Encoding($false).

8.3. Encoding UTF8 con BOM

Problema: Set-Content -Encoding UTF8 en PowerShell 5.1 agrega un BOM al inicio del archivo, causando que Zabbix Agent 2 no lea correctamente la configuración.

Solución: Usar [System.IO.File]::WriteAllLines() con UTF8Encoding($false) para escribir sin BOM.

```
$utf8NoBom = New-Object System.Text.UTF8Encoding $false
[System.IO.File]::WriteAllLines(
  "C:\Program Files\Zabbix Agent 2\zabbix_agent2.conf",
  $conf, $utf8NoBom)
```

9. Interpretación de los datos

Los valores de CPU representan el **tiempo acumulado de procesador en segundos** desde que el proceso se inició, no un porcentaje instantáneo.

- **Valores altos** indican procesos/usuarios con alto consumo acumulado de CPU.
- **RAM** muestra el consumo actual de memoria en MB.
- **Procs** indica la cantidad de procesos activos de ese usuario.

10. Notas adicionales

- El servicio Zabbix Agent 2 corre como **SYSTEM** por defecto, suficiente para ejecutar Get-Process -IncludeUserName.
- El intervalo de 5 minutos balancea visibilidad y carga del servidor. Se puede reducir a 1m si se requiere monitoreo más frecuente.

• Los items devuelven texto plano, no valores numéricos, por lo que **no se pueden graficar directamente**. Para gráficos se pueden crear scripts adicionales que devuelvan solo el valor numérico.

• La configuración debe replicarse en cada servidor donde se desee este monitoreo.

• Si el firewall bloquea la ejecución de PowerShell desde el agente, ejecutar `Set-ExecutionPolicy RemoteSigned` como administrador.